From MERN to GenAI Full-Stack Developer

A project-based roadmap (easy → hard) — Python back end, your existing React/Node skills for the front end

---

You already know how to build full-stack apps (React, Node, Express, MongoDB). The gap to close is the **AI/ML/GenAI layer**: Python fundamentals for data/ML, classic ML, deep learning, NLP, LLM APIs, prompt engineering, embeddings, vector databases, RAG, agents, and fine-tuning — then wiring all of that into full-stack apps the way you already know how to build them. The projects below are ordered so each one introduces one or two new concepts on top of what came before, instead of dumping everything at once. Build them roughly in order; skipping around is fine once you hit Phase 3.

Each project lists: what you're practicing, the concepts it forces you to learn, the tech stack, and what you should end up with (a working repo you can put on GitHub / your resume).

Phase 0 — Python & Data Foundations (1–2 weeks)

Skip anything you already know. The goal is just enough Python-for-data fluency to not get stuck later.

0.1. CSV Data Explorer (CLI tool)

Goal: Get comfortable with Python's data-handling libraries instead of JS equivalents.

Concepts covered: Python syntax vs JS, virtual envs & pip, NumPy arrays, Pandas DataFrames, basic plotting.

Stack: Python, Pandas, NumPy, Matplotlib

Deliverable: A CLI script that loads any CSV, prints summary stats, and outputs 2-3 charts.

0.2. REST API with FastAPI (port your Express instincts)

Goal: Map what you know from Express routes/middleware onto Python's most popular API framework, since it's what almost every AI backend uses.

Concepts covered: FastAPI routing, Pydantic models (like a JS schema validator), async endpoints, auto-generated OpenAPI docs.

Stack: Python, FastAPI, Uvicorn, Pydantic

Deliverable: A small CRUD API (e.g. notes or todos) with FastAPI, mirroring an Express API you'd already know how to write.

Phase 1 — Classic ML Fundamentals (2–3 weeks)

Before jumping to LLMs, build a real intuition for how models learn from data — it demystifies everything that comes later, including why LLMs behave the way they do.

1.1. House Price / Salary Predictor

Goal: Understand supervised learning end-to-end on a simple, tabular problem.

Concepts covered: Train/test split, linear & logistic regression, overfitting, evaluation metrics (MAE, RMSE, accuracy), feature scaling.

Stack: Python, scikit-learn, Pandas, Matplotlib

Deliverable: A script/notebook that trains a regression model on a public dataset and reports evaluation metrics.

1.2. Spam / Sentiment Classifier

Goal: Move from numeric features to text features — your first bridge into NLP.

Concepts covered: Text preprocessing, bag-of-words & TF-IDF, Naive Bayes / logistic regression classifiers, confusion matrix, precision/recall.

Stack: Python, scikit-learn, NLTK

Deliverable: A classifier (spam vs. not, or positive/negative sentiment) exposed via a FastAPI endpoint you built in Phase 0.

1.3. Handwritten Digit Recognizer

Goal: Get your first working neural network and understand what 'deep learning' actually means mechanically.

Concepts covered: Neural network basics, layers/weights/activation functions, backpropagation (conceptually), training loops, GPUs vs CPU.

Stack: Python, PyTorch or TensorFlow/Keras, MNIST dataset

Deliverable: A trained image classifier with a simple web UI (React) where a user draws a digit and gets a prediction.

Phase 2 — NLP, Embeddings & the Building Blocks of LLMs (2–3 weeks)

2.1. Semantic Search Engine over Your Own Notes/Docs

Goal: This is the single most important concept for GenAI work: turning text into vectors and searching by meaning, not keywords.

Concepts covered: Word/sentence embeddings, cosine similarity, vector search, chunking text.

Stack: Python, sentence-transformers (Hugging Face), NumPy/FAISS

Deliverable: A CLI or FastAPI service: paste a query, get back the most semantically similar chunks from a folder of your own documents.

2.2. Store Embeddings in a Real Vector Database

Goal: Move from an in-memory FAISS index to infrastructure you'll actually use in production GenAI apps.

Concepts covered: Vector databases, indexing strategies, metadata filtering, similarity thresholds.

Stack: Python, Chroma or Pinecone or Qdrant, FastAPI

Deliverable: Same semantic search as 2.1, but backed by a persistent vector DB with a simple React front end to query it.

2.3. Text Summarizer / Named-Entity Extractor

Goal: Practice using pretrained transformer models directly, without training your own.

Concepts covered: Hugging Face Transformers pipelines, pretrained model inference, tokenization, model cards/model selection.

Stack: Python, Hugging Face Transformers

Deliverable: A FastAPI endpoint that summarizes long text or extracts entities (names, dates, orgs) from pasted text or a PDF.

Phase 3 — Working with LLM APIs & Prompt Engineering (2 weeks)

3.1. AI Chat App (ChatGPT/Claude clone)

Goal: Your first true full-stack GenAI app — this is where your MERN skills plug straight in.

Concepts covered: LLM APIs (Anthropic/OpenAI), streaming responses, conversation/message history, system prompts, token limits & cost basics.

Stack: React front end, Node/Express or FastAPI back end, Anthropic or OpenAI API, MongoDB for chat history

Deliverable: A deployed chat interface with streaming replies and persisted conversation history, built on your existing MERN stack.

3.2. Prompt-Engineering Playground

Goal: Understand systematically what makes prompts reliable, instead of guessing.

Concepts covered: Zero-shot vs few-shot prompting, system vs user prompts, temperature/top-p, structured output (JSON mode), prompt templates.

Stack: React, FastAPI, Anthropic/OpenAI API

Deliverable: A tool where you can save prompt templates, run the same input against different prompt strategies, and compare outputs side by side.

3.3. Function-Calling / Tool-Use Assistant

Goal: Learn how LLMs take real actions instead of just returning text — the basis of every agent and copilot product.

Concepts covered: Tool/function calling, JSON schemas for tools, structured outputs, connecting an LLM to real APIs (e.g. weather, your own DB).

Stack: Python, FastAPI, Anthropic/OpenAI tool-use API

Deliverable: An assistant that can call at least 2 real tools (e.g. 'look up weather', 'query my database') based on user requests.

Phase 4 — Retrieval-Augmented Generation (RAG) (2–3 weeks)

4.1. 'Chat with Your PDF/Docs' App

Goal: Combine Phase 2 (embeddings/vector DB) with Phase 3 (LLM APIs) into the most common real-world GenAI pattern: RAG.

Concepts covered: RAG pipeline design, chunking strategies, retrieval + context injection, grounding answers in source documents, citations.

Stack: React, FastAPI, LangChain or LlamaIndex, vector DB (Chroma/Pinecone), Anthropic/OpenAI API

Deliverable: A deployed app: upload a PDF/set of docs, ask questions, get answers grounded in and citing the uploaded content.

4.2. RAG Evaluation & Guardrails Dashboard

Goal: Learn to measure whether a GenAI feature actually works and to keep it from doing anything unsafe.

Concepts covered: Evaluation metrics for RAG (relevance, faithfulness), hallucination detection basics, input/output guardrails, logging & observability.

Stack: Python, FastAPI, an eval framework (e.g. Ragas), MongoDB for logs

Deliverable: A dashboard showing query logs, retrieved chunks, and automated scores for how grounded each answer was.

Phase 5 — Agents & Multi-Step Workflows (2–3 weeks)

5.1. Autonomous Research/Task Agent

Goal: Move from single LLM calls to multi-step, self-directed workflows — the core of 'agentic AI' products.

Concepts covered: Agent loops (plan → act → observe), multi-tool orchestration, memory across steps, error handling/retries.

Stack: Python, LangGraph or CrewAI or a custom agent loop, Anthropic/OpenAI API

Deliverable: An agent that takes a goal (e.g. 'research X and write a summary report'), plans steps, uses tools, and produces a final result.

5.2. Multi-Agent App with a Full React Dashboard

Goal: Bring everything together into a portfolio-grade product: agents, RAG, tool use, and a polished MERN-style front end.

Concepts covered: Multi-agent coordination, WebSockets/streaming for live agent progress, task queues, full-stack state management for async AI workflows.

Stack: React, Node/Express or FastAPI, WebSockets, MongoDB, vector DB, Anthropic/OpenAI API

Deliverable: A deployed multi-agent product (e.g. AI research assistant, AI customer-support system, or AI code-review bot) with a live-updating dashboard.

Phase 6 — Fine-Tuning & Advanced Topics (2–3 weeks, optional but strong for resume)

6.1. Fine-Tune a Small Open Model

Goal: Understand what fine-tuning actually changes, and when it's the right tool vs. prompting/RAG.

Concepts covered: Fine-tuning vs. prompting vs. RAG (when to use which), LoRA/PEFT basics, dataset preparation, evaluation before/after.

Stack: Python, Hugging Face Transformers + PEFT, a small open-source model

Deliverable: A fine-tuned small model (e.g. for a narrow task like classifying support tickets) with before/after evaluation results.

6.2. Multimodal App (Image + Text)

Goal: Extend GenAI skills beyond text into vision, which is increasingly expected in full-stack GenAI roles.

Concepts covered: Multimodal model APIs, image input handling, combining vision + language outputs.

Stack: React, FastAPI, a multimodal LLM API (e.g. Claude/GPT-4 with vision)

Deliverable: An app where users upload an image (e.g. a receipt, whiteboard photo, or screenshot) and get structured, usable output.

6.3. Capstone: Production-Grade GenAI SaaS

Goal: Prove you can ship a real product, not just a demo — this is your strongest resume/portfolio piece.

Concepts covered: Auth, rate limiting & cost control, streaming UI, RAG + agents + tool use combined, deployment (Docker, CI/CD), monitoring/logging, prompt/version management.

Stack: Full MERN-equivalent stack + Python AI services, Docker, a cloud host (Render/AWS/Vercel), vector DB, Anthropic/OpenAI API

Deliverable: A live, deployed GenAI SaaS product with auth, billing-ready structure, and a real use case (e.g. AI meeting notes, AI support copilot, AI content generator).

How to Use This List

Don't wait until you 'finish learning' to start — build Phase 0 in parallel with reading about Phase 1. Push every project to GitHub with a short README explaining the concept it demonstrates; by the end of Phase 4 you'll already have a legitimate GenAI full-stack portfolio. Phases 5 and 6 are what separate a 'used an LLM API' resume from an 'AI/GenAI full-stack developer' resume.